

AI-Based Coding Assessment Platform

Problem Statement Document — Phase 1, Week 1

Project Area: Artificial Intelligence / Education / Full-Stack Development

Primary Focus: AI-assisted automated coding assessment and personalized learning support

1. Background

Programming education and assessment increasingly use online coding platforms to evaluate students through coding problems and automated test cases. These systems can quickly determine whether a submitted program produces the expected output, but a basic score or pass/fail result may provide limited information about code quality, problem-solving approach, individual skill gaps, or the next steps a student should take.

AI-assisted coding assessment is an important research direction because programming solutions contain both objective information, such as test-case results and execution behaviour, and qualitative information, such as readability, structure, possible inefficiencies, and explanation of logic. A well-designed assessment system can combine automated code evaluation with AI-assisted analysis without allowing AI output to replace objective execution results.

2. Problem Context

Traditional coding assessment commonly includes manually checked assignments, fixed coding questions, online judges, test-case based evaluation, and teacher-provided feedback. These approaches are useful, but they can become time-consuming for instructors and may provide limited personalized guidance when the number of students is large.

AI-based approaches can provide code explanations, hints, review comments, and learning recommendations. However, an AI system also introduces challenges such as unreliable generated feedback, excessive dependence on AI-generated solutions, privacy concerns, API cost and latency, and the need to keep objective code correctness separate from AI interpretation.

Therefore, the project should investigate how automated assessment and AI-assisted feedback can be combined into a practical student-focused platform.

3. Core Problem Statement

Existing coding assessment approaches can efficiently check program correctness, but their effectiveness as a complete learning and assessment system may be limited by fixed difficulty, limited personalized feedback, weak skill-level analysis, and separation between assessment results and learning recommendations. Therefore, there is a need for an AI-Based Coding Assessment Platform that systematically evaluates submitted programs using test cases, analyzes code with AI where appropriate, adapts assessment difficulty using performance information, and provides personalized feedback and learning recommendations.

4. Research Problem

The research problem is not simply to build an online coding judge. The project must investigate whether combining objective automated code evaluation with AI-assisted analysis can provide useful, understandable, and personalized assessment support while maintaining reliable correctness evaluation.

Challenge	Research concern
Limited personalized feedback	A score or test-case result may not explain why a solution can be improved or what the student should learn next.
Fixed assessment difficulty	The same difficulty for every learner may not accurately represent different skill levels or support progressive learning.
Skill identification	Individual strengths and weaknesses across programming topics may be difficult to identify from a single score.
AI reliability	AI-generated explanations or suggestions may be incomplete or incorrect and should not replace objective test-case results.
AI dependency	Students may rely on AI-generated solutions instead of developing their own problem-solving ability.
Scalability and performance	Code execution, database operations, analytics, and AI requests must work efficiently as the number of users and submissions increases.
Security	Untrusted student code and AI/API credentials must be handled in a controlled and secure manner.

5. Proposed Problem-Solving Direction

The proposed project will investigate a full-stack coding assessment pipeline in which a student selects an assessment, writes a solution in an online code editor, submits the program, and receives objective evaluation through controlled test cases. The assessment data can then be used by an AI component to generate code review, explanations, hints, and personalized learning suggestions.

The platform will be developed using the MERN stack. Krutrim Cloud/Qwen is proposed for AI-assisted analysis. The AI layer will be treated as a supporting component: correctness will primarily come from controlled code execution and test-case evaluation, while AI will interpret results and provide educational feedback.

Proposed high-level workflow:

Student → Login/Assessment → Coding Editor → Code Submission → Code Execution & Test Cases → Score/Results → AI Code Analysis → Feedback/Hints → Skill Analysis → Personalized Recommendation

6. Project Objectives

1. Study existing coding assessment and AI-assisted programming assessment approaches through selected research literature.
2. Identify important requirements for reliable automated coding assessment, including test cases, execution limits, scoring, and result handling.
3. Design and implement a web-based coding assessment platform using the MERN stack.
4. Provide an online coding editor and automated evaluation of submitted solutions.
5. Integrate an AI service such as Krutrim Cloud/Qwen for code review, explanations, hints, and improvement suggestions where appropriate.
6. Develop performance and skill analysis using assessment history and topic-wise results.
7. Investigate adaptive assessment in which question difficulty or topic selection can respond to student performance.
8. Evaluate the platform using correctness, AI feedback quality, processing time, usability, reliability, and comparison with a basic automated assessment workflow.

7. Scope of the Problem

In scope: user authentication and roles, coding problem management, online code editor, code submission, controlled code execution, test-case evaluation, scoring, AI-assisted code review and explanation, progressive hints, adaptive assessment, skill analysis, personalized recommendations, assessment history, teacher/admin analytics, REST APIs, MongoDB storage, and a web-based demonstration interface.

Initially out of scope: training a foundation AI model from scratch, unrestricted execution of arbitrary system commands, guaranteed detection of every cheating method, automatic browsing of unknown websites, production-scale infrastructure beyond project requirements, and treating AI-generated feedback as the sole authority for correctness. These may be considered future extensions.

Research note: The problem formulation is intentionally broader than a single AI model or programming language. The team should use its selected literature and project experiments to justify the final problem categories, assessment rules, AI prompting approach, execution method, metrics, and technology choices.